

Lista Encadeada Simples

Resumo completo com teoria e implementação em C

@resumoscc · Ciência da Computação · UFSJ

O que é uma Lista Encadeada?

Uma lista encadeada é uma estrutura de dados em que os elementos — chamados de nós (ou células) — são armazenados em posições não necessariamente contíguas na memória. Ao contrário de um vetor, onde os elementos ficam lado a lado, em uma lista encadeada cada nó sabe apenas onde está o próximo elemento.

Pensa assim: é como uma corrente. Cada elo só está conectado ao próximo. Se você quiser chegar ao quinto elo, precisa passar pelo primeiro, segundo, terceiro e quarto — não tem atalho.

Por que usar listas encadeadas?

Em vetores, inserir ou remover elementos do meio é caro — você precisa deslocar vários elementos.

Na lista encadeada, basta redirecionar alguns ponteiros.

A desvantagem é que não há acesso direto por índice — você sempre percorre do início.

Estrutura da Lista

Uma lista encadeada simples com célula cabeça é composta por:

- Célula cabeça (ou nó sentinela): um nó especial no início que não guarda dado útil. Simplifica as operações de inserção e remoção.
- Nós de dados: cada nó guarda um item e um ponteiro para o próximo nó.
- Ponteiro Primeiro: aponta sempre para a célula cabeça.
- Ponteiro Último: aponta sempre para o último nó da lista.

Visualmente, a lista fica assim:

```
Primeiro                                Ultimo
|                                     |
[cabeça] --> [item1] --> [item2] --> [item3] --> NULL
```

No código em C, isso é representado da seguinte forma:

```
typedef struct {
    int Chave;          // dado armazenado no nó
} TipoItem;

typedef struct TipoCelula *TipoApontador;

typedef struct TipoCelula {
    TipoItem      Item;    // o dado
    TipoApontador Prox;    // ponteiro para o próximo nó
} TipoCelula;

typedef struct {
    TipoApontador Primeiro; // aponta para a célula cabeça
    TipoApontador Ultimo;   // aponta para o último nó
} TipoLista;
```

Funções Básicas

FLVazia — Inicializar a lista

Cria a célula cabeça e faz Primeiro e Ultimo apontarem para ela. A lista começa vazia.

```
void FLVazia(TipoLista *Lista) {  
    Lista->Primeiro = malloc(sizeof(TipoCelula));  
    Lista->Ultimo    = Lista->Primeiro;  
    Lista->Primeiro->Prox = NULL;  
}
```

Como funciona

Aloca memória para a célula cabeça.

Faz Primeiro e Ultimo apontarem para ela.

Prox da cabeça recebe NULL — não há elementos ainda.

Vazia — Verificar se a lista está vazia

A lista está vazia quando Primeiro e Ultimo apontam para o mesmo lugar — ou seja, só existe a célula cabeça, sem nenhum dado.

```
int Vazia(TipoLista Lista) {  
    return (Lista.Primeiro == Lista.Ultimo);  
}
```

Inserir — Inserir no final

Cria um novo nó com o item recebido e o adiciona após o último nó da lista. O ponteiro Ultimo é atualizado para apontar para esse novo nó.

```
void Insere(TipoItem x, TipoLista *Lista) {
    Lista->Ultimo->Prox = malloc(sizeof(TipoCelula));
    Lista->Ultimo        = Lista->Ultimo->Prox;
    Lista->Ultimo->Item = x;
    Lista->Ultimo->Prox = NULL;
}
```

Passo a passo

1. Aloca um novo nó e conecta ao Prox do atual Ultimo.
2. Atualiza Ultimo para apontar para o novo nó.
3. Copia o item para o novo nó.
4. Prox do novo nó recebe NULL (é o último).

Retira — Remove um nó específico

Remove o nó que vem APÓS o ponteiro p passado como parâmetro. Isso é necessário porque em listas encadeadas simples não temos acesso direto ao nó anterior — então passamos o nó que está antes do que queremos remover.

```
void Retira(TipoApontador p, TipoLista *Lista, TipoItem *Item) {
    TipoApontador q;
    if (Vazia(*Lista) || p == NULL || p->Prox == NULL) {
        printf("Erro\n"); return;
    }
    q = p->Prox; // nó a ser removido
    *Item = q->Item; // salva o item
    p->Prox = q->Prox; // pula o nó removido
    if (p->Prox == NULL) // se era o último,
        Lista->Ultimo = p; // atualiza Ultimo
    free(q); // libera memória
}
```

Imprime — Percorrer e imprimir a lista

Percorre a lista do início ao fim, imprimindo o valor de cada nó. Começa pelo primeiro nó de dados (pula a célula cabeça).

```
void Imprime(TipoLista Lista) {
    TipoApontador Aux = Lista.Primeiro->Prox; // pula a cabeça
    while (Aux != NULL) {
        printf("%d\n", Aux->Item.Chave);
        Aux = Aux->Prox;
    }
}
```

Funções Adicionais

Inserelnicio — Inserir no início

Insere um novo nó logo após a célula cabeça, ou seja, no início da lista de dados. Os outros elementos permanecem onde estão.

```
void Inserelnicio(TipoItem x, TipoLista *Lista) {
    if (Vazia(*Lista)) {
        Insere(x, Lista); // se vazia, usa Insere normal
        return;
    }
    TipoApontador p = malloc(sizeof(TipoCelula));
    p->Item = x;
    p->Prox = Lista->Primeiro->Prox; // aponta para o 1º dado
    Lista->Primeiro->Prox = p;       // cabeça aponta para p
}
```

Atenção

Se a lista estiver vazia, Inserelnicio chama a função Insere normal.

Se não estiver vazia, o novo nó é encaixado entre a cabeça e o primeiro elemento.

RetiraUltimo — Remover o último elemento

Remove o último nó da lista e devolve o item armazenado nele. Como não temos ponteiro para o penúltimo, é necessário percorrer a lista toda para encontrá-lo.

```

void RetiraUltimo(TipoLista *Lista, TipoItem *Item) {
    if (Vazia(*Lista)) { printf("Erro\n"); return; }

    // caso especial: só um elemento
    if (Lista->Primeiro->Prox->Prox == NULL) {
        *Item = Lista->Primeiro->Prox->Item;
        TipoApontador q = Lista->Primeiro->Prox;
        Lista->Primeiro->Prox = NULL;
        Lista->Ultimo = Lista->Primeiro;
        free(q); return;
    }

    // caso geral: percorre até o penúltimo
    TipoApontador p = Lista->Primeiro->Prox;
    while (p->Prox->Prox != NULL)
        p = p->Prox;

    *Item = p->Prox->Item;
    TipoApontador q = p->Prox;
    p->Prox = NULL;
    Lista->Ultimo = p;
    free(q);
}

```

Por que precisa percorrer tudo?

Em listas encadeadas simples, cada nó só conhece o próximo.

Para remover o último, precisamos do penúltimo — e chegar até ele exige percorrer desde o início.

Isso torna RetiraUltimo uma operação $O(n)$.

Tamanho — Contar os elementos

Percorre toda a lista e conta quantos nós de dados existem (não conta a célula cabeça).


```

int Tamanho(TipoLista *Lista) {
    if (Vazia(*Lista)) return 0;
    TipoApontador p = Lista->Primeiro->Prox;
    int Contador = 0;
    while (p != NULL) {
        p = p->Prox;
        Contador++;
    }
    return Contador;
}

```

Inverte — Inverter a lista

Inverte a ordem de todos os nós da lista. Usa três ponteiros auxiliares para não perder o fio durante a inversão dos links.

```

void Inverte(TipoLista *Lista) {
    if (Vazia(*Lista)) { printf("Erro\n"); return; }

    TipoApontador Anterior = NULL;
    TipoApontador Atual = Lista->Primeiro->Prox;
    TipoApontador Proximo;
    TipoApontador NovoUltimo = Lista->Primeiro->Prox; // será o último

    while (Atual != NULL) {
        Proximo = Atual->Prox; // salva o próximo
        Atual->Prox = Anterior; // inverte o link
        Anterior = Atual; // avança Anterior
        Atual = Proximo; // avança Atual
    }
    Lista->Primeiro->Prox = Anterior; // novo primeiro
    Lista->Ultimo = NovoUltimo; // novo último
}

```

Como a inversão funciona

A cada iteração, o link Prox do nó atual é redirecionado para o nó anterior.

Isso vai "virando" a corrente de trás para frente.

No fim, o que era o primeiro vira o último, e vice-versa.

Resumo das Funções

Função	O que faz	Complexidade
FlVazia	Inicializa a lista com célula cabeça	$O(1)$
Vazia	Verifica se a lista está vazia	$O(1)$
Inserir	Inserir no final da lista	$O(1)$
InserirInicio	Inserir no início da lista	$O(1)$
Retira	Remove o nó após o ponteiro p	$O(1)$
RetiraUltimo	Remove o último nó	$O(n)$
Tamanho	Retorna o número de elementos	$O(n)$
Inverte	Inverte a ordem da lista	$O(n)$
Imprime	Imprime todos os elementos	$O(n)$

Pontos importantes para a prova

- A célula cabeça nunca guarda dados — ela é só um nó auxiliar que simplifica as operações.
- A lista está vazia quando Primeiro == Ultimo (ambos apontam para a cabeça).
- Inserir e InserirInicio são $O(1)$ porque não precisam percorrer a lista.
- RetiraUltimo, Tamanho e Inverte são $O(n)$ porque percorrem a lista toda.
- Na função Retira, você passa o nó ANTERIOR ao que quer remover — não o nó em si.
- Sempre libere memória com free() ao remover um nó para evitar vazamento de memória.

Dica de active recall

Feche esse resumo e tente responder:

1. O que é a célula cabeça e por que ela existe?
2. Como a função Inverte funciona passo a passo?
3. Por que Retira recebe o nó anterior e não o nó a ser removido?
4. Qual a diferença entre Insere e InseremInicio?